

COMPUTATIONAL FOUNDATIONS FOR ARTIFICIAL INTELLIGENCE



Project Title: Hospital bed allocation system

Section: 12

Batch: 15

2520030012 Dharmika
2520030211 Bhaarathi
2520090232 Sahithya

Under the guidance of

Dr. Ramakanth Komati

Term -3 semester 2025-2026



Project Abstract



System Framework

- **AI-Driven Management:** Replaces manual, error-prone, and time-consuming allocation pipelines.
- **Dynamic Prioritization:** Automates live bed assignments based on clinical severity levels.
- **Multi-Ward Coverage:** Efficiently coordinates ICU rooms, emergency zones, and general ward beds.



Technical Blueprint

- **Algorithmic Search:** Uses Constraint Satisfaction and AI search routines to completely eliminate allocation conflicts.
- **Data Infrastructure:** Programmed in Python using optimized data structures (lists, dictionaries, heaps).
- **Future-Ready:** Scalable architecture primed for ML modules, cloud hosting, and continuous live tracking.



Strategic Impact

- **Optimized Logistics:** Dramatically increases resource utilization indices across complex hospital networks.
- **Critical Care Speed:** Drastically decreases emergency triage response times when beds are scarce.
- **Operational Trust:** Minimizes human workflow efforts while establishing transparent decision traces.

Technology or Concepts that are used			
CO's	Technology or Concept Used	Reason	Why it is used
CO-1	- Problem Formulation & Python Data Structures - Rule-Based Logic & Constraints - Trace Logging & Testing	Registration forms, validation, Python/React logic, data structures, state handling	- Efficient healthcare data management - Secure and accurate patient data entry - Easy debugging and system testing
CO-2	- Search Algorithms Concept - Heuristic & Efficient Search Concepts	Simple searching/filtering of patients resembles search concepts, but not full BFS/A* algorithms	- Quick patient information retrieval - Faster and efficient data access
CO-3	- CSP (Constraint Satisfaction Problems) - Constraint Propagation	Appointment scheduling and validation use constraint-based logic	- Prevents scheduling conflicts - Manages doctor availability efficiently
CO-4	- Decision Logic - Utility Functions	Decision-making exists in navigation and role access, but not actual minimax/game AI	- Controls user access securely - Improves healthcare management efficiency
CO-5	- Bayesian/Probability Concepts - Uncertainty Handling	Probability, Bayesian networks, HMMs are generally not present unless you added prediction features	- Supports smart healthcare decisions - Ensures reliable system performance

CO's	Technology or Concept Used	Reason	Why it is used
CO-6	- Hybrid AI Architecture - Explainable Reasoning & Performance - Ethics & Limitations	Integration of modules, validation, security, explainability, performance handling	- Integrates multiple system modules - Improves user experience and maintenance - Ensures secure patient data handling

```

# CO1: Problem Formulation (Overall system initialization)
# Main Program execution starts here
hospital = HospitalBedAllocation() # Initialize the hospital system

# CO1: State Representation (Adding beds to define initial state)
# Add various types of beds to the hospital (increased quantity and new Observation beds)
hospital.add_bed(Bed(101, "ICU"))
hospital.add_bed(Bed(102, "General"))
hospital.add_bed(Bed(103, "Emergency"))
hospital.add_bed(Bed(104, "ICU"))
hospital.add_bed(Bed(105, "General"))
hospital.add_bed(Bed(106, "Emergency"))
hospital.add_bed(Bed(107, "General"))
hospital.add_bed(Bed(201, "Observation")) # New Observation bed
hospital.add_bed(Bed(202, "Observation")) # Another Observation bed
hospital.add_bed(Bed(203, "Observation")) # Another Observation bed

```

CO-1

```

# CO1: Python Functions
# CO3: Scheduling/Timetabling
# CO5: Markov chains & HMM intuition for tracking
# Method to discharge patients whose duration of stay has ended
def discharge_patients(self):
    discharged_today = [] # List to temporarily store patients discharged today
    beds_to_free = [] # List to temporarily store bed IDs to be freed

    # Find patients to discharge by iterating through current allocations
    for bed_id, patient in self.allocations.items():
        # Check if patient has been admitted and their duration of stay is complete
        if patient.admission_day != -1 and (self.current_day - patient.admission_day) >= patient.duration_of_stay:
            discharged_today.append(patient)
            beds_to_free.append(bed_id)

    # Process discharges if any patients are to be discharged
    if discharged_today:
        print("\n--- Patients Discharged Today ---")
        for bed_id in beds_to_free:
            patient = self.allocations.pop(bed_id) # Remove patient from allocations
            for bed in self.beds:
                if bed.bed_id == bed_id:
                    bed.is_occupied = False # Mark bed as free
                    print(f" Patient {patient.name} (ID: {patient.patient_id}) discharged from Bed {bed.bed_id} ({bed.bed_type})")
                    break
            else:
                print(" No patients discharged today.")

```

CO-3,5

```

# CO1: Python Functions, Rule Sets
# CO2: Greedy Search
# CO4: Bounded Rationality
# Method to allocate a bed to the highest priority patient
def allocate_bed(self):
    if self.patient_queue.empty():
        print("No patients waiting")
        return

    priority, patient = self.patient_queue.get() # Get the highest priority patient

    # Logic for low-severity patients (less than 5) to be allocated to Observation beds first
    if patient.severity < 5:
        for bed in self.beds:
            if not bed.is_occupied and bed.bed_type == "Observation":
                bed.is_occupied = True
                patient.admission_day = self.current_day # Set admission day
                self.allocations[bed.bed_id] = patient # Store patient object in allocation
                print("\nBed Allocated Successfully (Observation)")
                print(patient)
                print(bed)
                return
        print(f"No available Observation bed for low-severity patient {patient.name}")

    # Existing logic for other patients (severity >= 5 or no Observation bed available)
    for bed in self.beds:
        if not bed.is_occupied and bed.bed_type == patient.required_bed:
            bed.is_occupied = True
            patient.admission_day = self.current_day # Set admission day
            self.allocations[bed.bed_id] = patient # Store patient object in allocation

            print("\nBed Allocated Successfully")
            print(patient)
            print(bed)
            return

```

CO-2,4

```

# CO6: Explainable Reasoning Traces
# Display detailed status of available beds
print("\n--- Detailed Bed Status (Available) ---")
bed_status = hospital.display_beds() # Get structured bed status data
if bed_status['available_details']:
    for line in bed_status['available_details']:
        print(line)
else:
    print(" No beds currently available.")

```

CO-6



Project Strategic Framework



EXISTING PROBLEMS

- Manual allocation is prone to human error
- Critical emergency bed delays
- Acute shortage of ICU visibility
- Operational bottlenecks during peak loads



PROJECT MOTIVATION

- Enhance systemic hospital efficiency
- Minimize patient triage waiting time
- Deploy intelligent resource mapping
- Bolster critical care support systems



THE AI PROPOSED SOLUTION

- Real-time automated bed tracking
- Severity-based priority assignment
- Algorithmic AI search & allocation
- Faster, data-validated medical decisions



CURRENT ALTERNATIVES

- Physical manual log registers
- Static spreadsheet (Excel) tracking
- Legacy non-intelligent databases

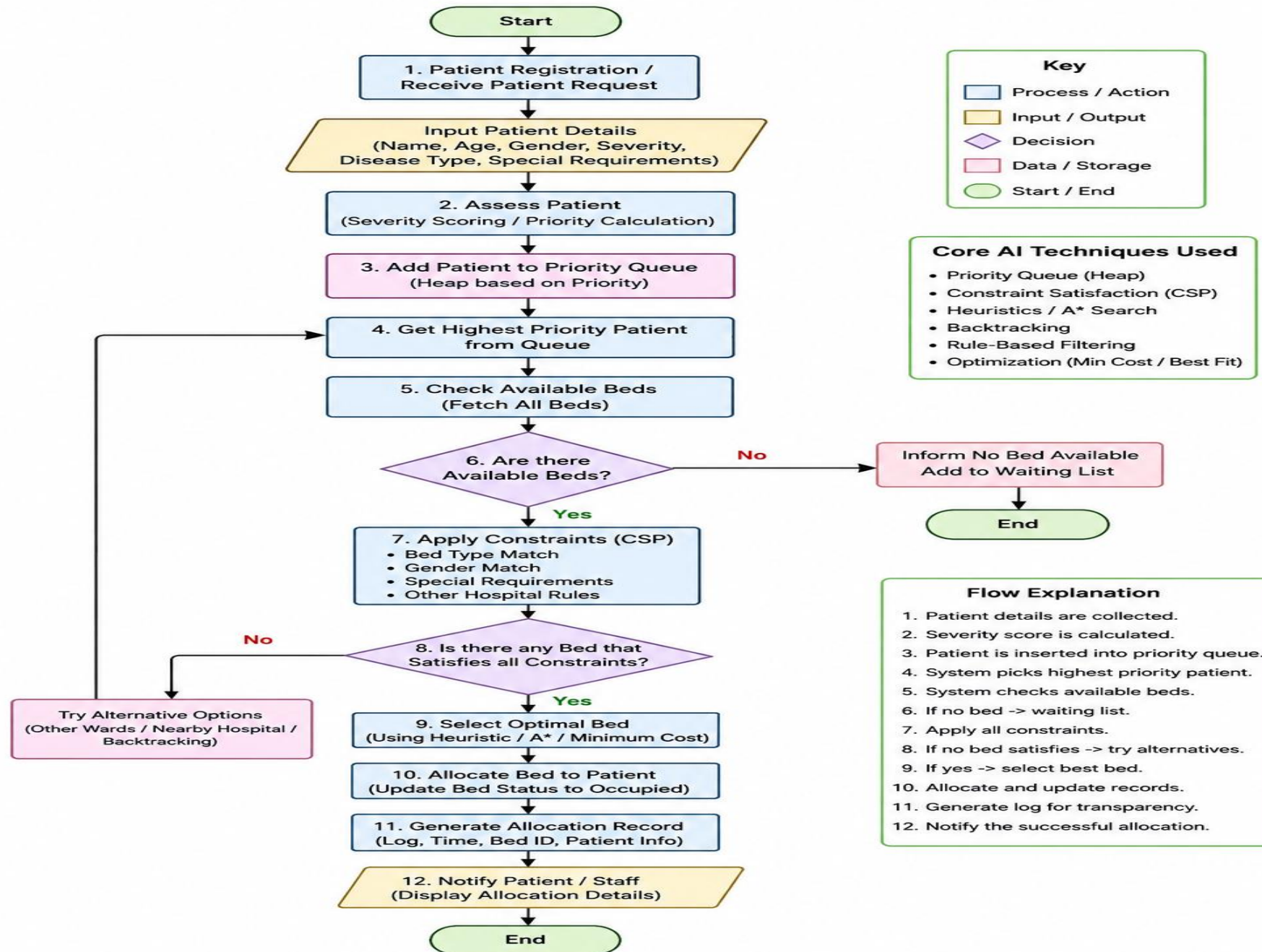


STRATEGIC ADVANTAGE

- Near-instant processing speed
- Superior resource allocation accuracy
- Elimination of manual prioritization bias
- Substantial reduction in human error

Designed for optimal healthcare resource management.

Hospital Bed Allocation System – Flowchart



Conclusion & Strategic Impact

The AI-Based Hospital Bed Allocation System successfully transitions healthcare logistics from **error-prone manual processes** to a **high-performance algorithmic engine**, ensuring critical resources are optimized for maximum clinical impact.



Operational Excellence

The system eliminates allocation conflicts and data entry latency, providing real-time visibility that is essential during peak patient surges or medical emergencies.



Clinical Priority

By leveraging AI search and priority queues, high-severity cases are automatically front-loaded for ICU and emergency beds, directly supporting life-saving interventions.



Future Readiness

The modular Python architecture is fully primed for future integration with Predictive ML models, Cloud infrastructure, and real-time hospital monitoring IoT networks.



THANK YOU

FOR YOUR TIME AND ATTENTION!



Better Records • Better Care • Better Health

